

Backend Work Spec: Add New SKU

Feature: an 'Add SKU bundle' action on the frontend SKU-bundles page. The UI is built and mock-backed; this document specifies the one backend endpoint required to make it work against the live API. Owner: core/backend track.

1. What the frontend already does

The SKU-bundles page has a '+ Add SKU bundle' button that opens a modal form. On submit it calls:

```
POST /skus      (client: api.proposed.createSku)
Content-Type: application/json
```

Today this route does not exist in openapi/openapi.json, so a live backend returns 405. The frontend already validates the request and response with zod, surfaces errors in the modal, and refreshes the list on success. Nothing else on the frontend needs to change once the endpoint exists.

2. Endpoint to implement

POST /skus

Create one new SKU bundle from its declarative config (build phase: 'Define SKU'). Returns the created SkuConfig.

Request body - CreateSkuRequest

```
{
  "sku_id":      "bracket-a",          // required,^[a-z0-9][a-z0-9_-]*$
  "name":        "Mounting Bracket A", // optional
  "result_type": "detection",          // required: detection|classification|measurement
  "adapter_id":  "rfdetr_bracket_a",   // required
  "plugin_id":   "bracket_a_rules",    // required
  "classes":     ["screw", "crack"],    // optional
  "thresholds":  { "screw": 0.5 },     // optional (map<string,number>)
  "params":      {}                   // optional (freeform)
}
```

Success - 201 Created, body = SkuConfig

```
{
  "sku_id": "bracket-a", "name": "Mounting Bracket A",
  "result_type": "detection",
  "adapter_id": "rfdetr_bracket_a", "plugin_id": "bracket_a_rules",
  "classes": ["screw","crack"], "thresholds": {"screw":0.5}, "params": {}
}
```

Errors

- 409 Conflict - a bundle with that sku_id already exists.
- 422 Unprocessable - missing/invalid field (reuse the existing FastAPI validation error shape).

3. What the handler must do

1. Validate the body into a CreateSkuRequest / SkuConfig pydantic model (reuse the frozen SkuConfig from core/contracts.py; add a small CreateSkuRequest or accept SkuConfig directly).
2. Reject if skus/<sku_id>/ already exists (409).
3. Scaffold the bundle on disk:

```
skus/<sku_id>/
config.yaml    <- write the SkuConfig (result_type, adapter_id,
                  plugin_id, classes, thresholds, params)
```

```
data/          <- empty (created)
model/         <- empty (created)
metrics/       <- empty (created)
sop.yaml       <- optional stub, if SOP authoring lands too
```

4. Register / refresh the registry so GET /skus and GET /skus/{id} immediately return the new bundle (the registry already discovers bundles; ensure a create triggers a rescan or an in-place add).
5. Return 201 with the persisted SkuConfig.

4. Constraints (from CLAUDE.md)

- No branching on sku_id in shared code. This endpoint is generic: it writes data and registers - it must not special-case any SKU.
- Do NOT resolve/instantiate the adapter or plugin at create time. adapter_id / plugin_id are stored as ids; they are resolved later at inspect time. A create with an unknown adapter/plugin id is allowed (the bundle simply is not runnable until they exist).
- SkuConfig stays declarative data only - no executable logic.

5. Explicitly out of scope for this endpoint

Adding a SKU end-to-end is: define config (this endpoint) + author SOP + capture data + annotate + train + drop weights + implement adapter + implement rule plugin. This endpoint only does the FIRST step - it defines the bundle config and registers it. The rest is added out-of-band, exactly as the PRD's SKU onboarding flow describes.

6. Publish it in the schema

After implementing, regenerate openapi/openapi.json (python -m scripts.publish_openapi). The frontend will then move createSku from its 'proposed' section to the authoritative mirror. Add roughly:

```
paths:
  /skus:
    post:
      summary: Create a SKU bundle
      requestBody: {required: true, content: {application/json:
        {schema: {$ref: "#/components/schemas/CreateSkuRequest"}}}}
      responses:
        "201": {content: {application/json:
          {schema: {$ref: "#/components/schemas/SkuConfig"}}}}
        "409": {description: SKU already exists}
        "422": {description: Validation error}
```

7. Acceptance criteria

- POST /skus with a valid body creates skus/<id>/config.yaml and returns 201 + SkuConfig.
- The new SKU appears in GET /skus and GET /skus/{id} without a restart.
- POST with an existing id returns 409; invalid body returns 422.
- Frontend: with VITE_USE MOCKS=false, the 'Add SKU bundle' modal creates a real bundle and the new card renders. (Already works with mocks on.)
- A conformance/backend test covers create + list-reflects-new + duplicate 409.